

THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD



COURSE TITLE	Database Systems
COURSE CODE	CS-2204
LAB TITLE	Open Ended Lab
STUDENT NAME	MAHDI ALI NAHVI
STUDENT ROLL NUMBER	2024-SE-16
INSTRUCTOR NAME	Engr. Awais Rathore
SUBMISSION DATE	September 21, 2026
LAB NUMBER	13

Task 1 — System Architecture & Entity-Relationship Design

CarGo Rentals needed to replace spreadsheet-based tracking of customers, vehicles, rentals, and payments with a relational database that prevents duplicate/inconsistent records and enforces the rule that a vehicle cannot be booked while it is already rented.

1.1 Entities Identified

- Customers — people who rent vehicles. Needs a unique identifier, contact details, and an identity document for verification.
- Vehicles — the fleet available for rent. Needs a plate number, model details, a daily rate, and a live availability status.
- Rentals — the transactional link between a customer and a vehicle for a period of time. This is the entity where the business rule (no double booking) must be enforced.
- Payments — money collected against a rental. Modeled separately from Rentals because a single rental may receive more than one payment (e.g., an advance and a final settlement).

1.2 Entity-Relationship Design

Relationships:

- Customers (1) — (N) Rentals: one customer can make many rentals over time.
- Vehicles (1) — (N) Rentals: one vehicle can be rented many times (at different, non-overlapping periods).
- Rentals (1) — (N) Payments: one rental can be settled through one or more payments.

Textual ER summary:

```
Customers (CustomerID PK)
|
| 1---N
v
Rentals (RentalID PK, CustomerID FK, VehicleID FK)
^
| N---1
|
Vehicles (VehicleID PK)

Rentals (1) ----N----> Payments (PaymentID PK, RentalID FK)
```

1.3 Key & Constraint Justification

Table	Primary Key	Foreign Key(s)	Key Constraints
Customers	CustomerID (surrogate, AUTO_INCREMENT)	—	Phone UNIQUE, CNIC UNIQUE, Email UNIQUE, NOT NULL on identifying fields
Vehicles	VehicleID (surrogate)	—	VehicleNumber UNIQUE, DailyRate CHECK (> 0), Status ENUM default 'Available'
Rentals	RentalID (surrogate)	CustomerID → Customers, VehicleID → Vehicles	CHECK (ScheduledReturnDate >= RentalDate); ON DELETE RESTRICT so a customer/vehicle with rental history cannot be silently deleted

Payments	PaymentID (surrogate)	RentalID → Rentals	AmountPaid CHECK (> 0); ON DELETE CASCADE so payments are cleaned up if a rental record is removed
----------	-----------------------	--------------------	---

A surrogate integer key (AUTO_INCREMENT) was used for every table instead of a natural key, because natural candidates such as phone number or vehicle number can change (a customer changes their number; a vehicle's plate can theoretically be reissued) — the surrogate key keeps foreign-key relationships stable. Natural keys are still protected using UNIQUE constraints so duplicate customers/vehicles cannot be inserted.

The core business rule — "a vehicle must not be available for another rental while it is currently rented" — is enforced redundantly at two levels for defense in depth: (1) a Vehicles.Status column that is only ever 'Available' or 'Rented' (or 'Maintenance'), and (2) a BEFORE INSERT trigger on Rentals (Task 5) that rejects any new rental row referencing a vehicle whose Status is not 'Available'. This means the rule holds even for direct SQL inserts that bypass application code.

Task 2 — Normalization (UNF → 1NF → 2NF → 3NF)

The company's original spreadsheet stored one flat record per rental:

RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000	2026-09-01	2026-09-04	15000

2.1 Unnormalized Form (UNF)

This single-table structure mixes facts about four different real-world things — the rental event, the customer, the vehicle, and the payment — into one row. Every attribute is atomic here (no repeating groups), but the design is still unnormalized in the practical sense that it repeats customer and vehicle data across every rental row for the same customer/vehicle.

2.2 Functional Dependencies Identified

- RentalID → CustomerName, CustomerPhone, VehicleNumber, VehicleModel, DailyRate, RentalDate, ReturnDate, PaymentAmount
- CustomerPhone → CustomerName (a phone number identifies one customer)
- VehicleNumber → VehicleModel, DailyRate (a plate number identifies one vehicle and its rate)

2.3 First Normal Form (1NF)

Requirement: every attribute must hold a single atomic value, and rows must be uniquely identifiable — no repeating groups. The original structure already stores atomic values per column, so 1NF is satisfied by formally designating RentalID as the primary key of a single flat relation:

Rentals_1NF (RentalID PK, CustomerName, CustomerPhone, VehicleNumber, VehicleModel, DailyRate, RentalDate, ReturnDate, PaymentAmount)

Anomalies still present at 1NF:

- Insertion anomaly: a new customer cannot be recorded until they actually make a rental (there is no rental-independent place to store customer data).
- Update anomaly: if Ali Khan's phone number changes, every historical rental row for Ali Khan must be updated, or the data becomes inconsistent.
- Deletion anomaly: deleting the only rental row for a customer erases all record that the customer ever existed.

2.4 Second Normal Form (2NF)

Requirement: the table must be in 1NF, and every non-key attribute must depend on the whole primary key, not part of it. Because RentalID here is a single-column key (not composite), there is no partial-dependency

violation to remove mechanically — but $\text{CustomerPhone} \rightarrow \text{CustomerName}$ and $\text{VehicleNumber} \rightarrow \text{VehicleModel}$, DailyRate are dependencies on non-key attributes, which is really a transitive-dependency problem carried into 2NF. To move meaningfully toward a well-structured design, we separate out the entities whose attributes depend on something other than RentalID :

```
Customers_2NF (CustomerPhone PK, CustomerName)
Vehicles_2NF (VehicleNumber PK, VehicleModel, DailyRate)
Rentals_2NF (RentalID PK, CustomerPhone FK, VehicleNumber FK,
            RentalDate, ReturnDate, PaymentAmount)
```

Anomalies resolved at 2NF: customer and vehicle data can now be inserted independently of a rental (fixes the insertion anomaly), and each customer's name / each vehicle's rate is stored exactly once (fixes the earlier update anomaly for those attributes).

Anomalies still present: PaymentAmount is functionally dependent on the rental period and the vehicle's DailyRate ($\text{PaymentAmount} \approx \text{DailyRate} \times \text{number of days}$), not purely on RentalID as an independent fact — a transitive dependency that 3NF addresses, and using CustomerPhone as a natural key is fragile if a customer's number changes.

2.5 Third Normal Form (3NF)

Requirement: the table must be in 2NF, and no non-key attribute may depend transitively on the primary key through another non-key attribute. We finalize the design by (a) replacing natural keys with surrogate IDs for stability, and (b) separating Payments from Rentals , since PaymentAmount is a fact about a payment event, not an intrinsic attribute of the rental row itself:

```
Customers_3NF (CustomerID PK, CustomerName, CustomerPhone UNIQUE, ...)
Vehicles_3NF (VehicleID PK, VehicleNumber UNIQUE, VehicleModel, DailyRate, Status)
Rentals_3NF (RentalID PK, CustomerID FK, VehicleID FK,
            RentalDate, ScheduledReturnDate, ActualReturnDate, TotalAmount)
Payments_3NF (PaymentID PK, RentalID FK, PaymentDate, AmountPaid, PaymentMethod)
```

Anomalies resolved at 3NF: TotalAmount is stored once per rental as a computed/recorded fact rather than being tangled with vehicle rate updates; a vehicle's DailyRate can change over time without corrupting historical rental charges, since Rentals stores the rate/amount that applied at the time; and payments can be added, refunded, or split across multiple transactions without touching the Rentals row. This 3NF structure is exactly what was implemented in `car_rental.sql`.

Task 3 — SQL JOIN Queries

Query 1 — INNER JOIN: Rental Details

Business question: customer name, vehicle number, vehicle model, rental date, and return date for every rental.

```
SELECT c.FullName AS CustomerName, v.VehicleNumber, v.Model AS VehicleModel,
       r.RentalDate, r.ActualReturnDate AS ReturnDate
FROM Rentals r
INNER JOIN Customers c ON r.CustomerID = c.CustomerID
INNER JOIN Vehicles v ON r.VehicleID = v.VehicleID
ORDER BY r.RentalDate;
```

Expected result: one row per rental (5 rows in the sample data), with `NULL` in ReturnDate for rentals that are still active.

Showing rows 0 - 4 (5 total, Query took 0.0016 seconds.) [RentalDate: 2026-08-20... - 2026-09-18...]

```
SELECT c.FullName AS CustomerName, v.VehicleNumber AS VehicleNumber, v.Model AS VehicleModel, r.RentalDate, r.ActualReturnDate AS ReturnDate FROM Rentals r INNER JOIN Customers c ON r.CustomerID = c.CustomerID INNER JOIN Vehicles v ON r.VehicleID = v.VehicleID ORDER BY r.RentalDate;
```

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
Sara Ahmed	LEA-456	Alto	2026-08-20	2026-08-22
Ali Khan	ABC-123	Corolla	2026-09-01	2026-09-04
Usman Tariq	LHR-654	City	2026-09-05	2026-09-07
Bilal Hussain	XYZ-789	Civic	2026-09-15	NULL
Ayesha Malik	KHI-321	Hilux	2026-09-18	NULL

Query 2 — LEFT JOIN: Customers and Their Rentals

Business question: all customers and the vehicles they have rented, including customers who have never rented.

```
SELECT c.CustomerID, c.FullName, v.VehicleNumber, v.Model AS VehicleModel, r.RentalDate
FROM Customers c
LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID
LEFT JOIN Vehicles v ON r.VehicleID = v.VehicleID
ORDER BY c.FullName;
```

Expected result: every customer appears at least once; a customer with no rentals would show NULLs for VehicleNumber, VehicleModel, and RentalDate.

Showing rows 0 - 4 (5 total, Query took 0.0012 seconds.) [FullName: ALI KHAN... - USMAN TARIQ...]

```
SELECT c.CustomerID, c.FullName, v.VehicleNumber, v.Model AS VehicleModel, r.RentalDate FROM Customers c LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID LEFT JOIN Vehicles v ON r.VehicleID = v.VehicleID ORDER BY c.FullName;
```

CustomerID	FullName	VehicleNumber	VehicleModel	RentalDate
1	Ali Khan	ABC-123	Corolla	2026-09-01
4	Ayesha Malik	KHI-321	Hilux	2026-09-18
3	Bilal Hussain	XYZ-789	Civic	2026-09-15
2	Sara Ahmed	LEA-456	Alto	2026-08-20
5	Usman Tariq	LHR-654	City	2026-09-05

Query 3 — LEFT JOIN: Vehicles and Current Rental Info

Business question: all vehicles and their current rental info, including vehicles that are not currently rented.

```
SELECT v.VehicleID, v.VehicleNumber, v.Model AS VehicleModel, v.Status AS VehicleStatus,
c.FullName AS CurrentRenter, r.RentalDate, r.ScheduledReturnDate
FROM Vehicles v
LEFT JOIN Rentals r ON v.VehicleID = r.VehicleID AND r.RentalStatus = 'Active'
LEFT JOIN Customers c ON r.CustomerID = c.CustomerID
ORDER BY v.VehicleNumber;
```

Expected result: every vehicle appears; vehicles that are Available show NULL for CurrentRenter and the rental dates.

Showing rows 0 - 5 (6 total, Query took 0.0363 seconds.) [VehicleNumber: ABC-123... - XYZ-789...]

```
SELECT v.VehicleID, v.VehicleNumber, v.Model AS VehicleModel, v.Status AS VehicleStatus, c.FullName AS CurrentRenter, r.RentalDate, r.ScheduledReturnDate FROM Vehicles v LEFT JOIN Rentals r ON v.VehicleID = r.VehicleID AND r.RentalStatus = 'Active' LEFT JOIN Customers c ON r.CustomerID = c.CustomerID ORDER BY v.VehicleNumber;
```

VehicleID	VehicleNumber	VehicleModel	VehicleStatus	CurrentRenter	RentalDate	ScheduledReturnDate
1	ABC-123	Corolla	Available	NULL	NULL	NULL
6	ISB-987	Sportage	Available	NULL	NULL	NULL
4	KHI-321	Hilux	Rented	Ayesha Malik	2026-09-18	2026-09-25
3	LEA-456	Alto	Available	NULL	NULL	NULL
5	LHR-654	City	Available	NULL	NULL	NULL
2	XYZ-789	Civic	Rented	Bilal Hussain	2026-09-15	2026-09-22

Query 4 — LEFT JOIN + GROUP BY + COUNT: Rentals per Customer

Business question: total number of rentals made by each customer, including customers with zero rentals.

```
SELECT c.CustomerID, c.FullName, COUNT(r.RentalID) AS TotalRentals
FROM Customers c
LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID
GROUP BY c.CustomerID, c.FullName
ORDER BY TotalRentals DESC;
```

Expected result: every customer appears exactly once; COUNT(r.RentalID) (not COUNT(*)) correctly returns 0 for customers with no matching rental rows, since COUNT of a column ignores NULLs produced by the LEFT JOIN.

Showing rows 0 - 4 (5 total, Query took 0.0303 seconds.)

```
SELECT c.CustomerID, c.FullName, COUNT(r.RentalID) AS TotalRentals FROM Customers c LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID GROUP BY c.CustomerID, c.FullName ORDER BY TotalRentals DESC;
```

CustomerID	FullName	TotalRentals
4	Ayesha Malik	1
1	Ali Khan	1
3	Bilal Hussain	1
5	Usman Tariq	1
2	Sara Ahmed	1

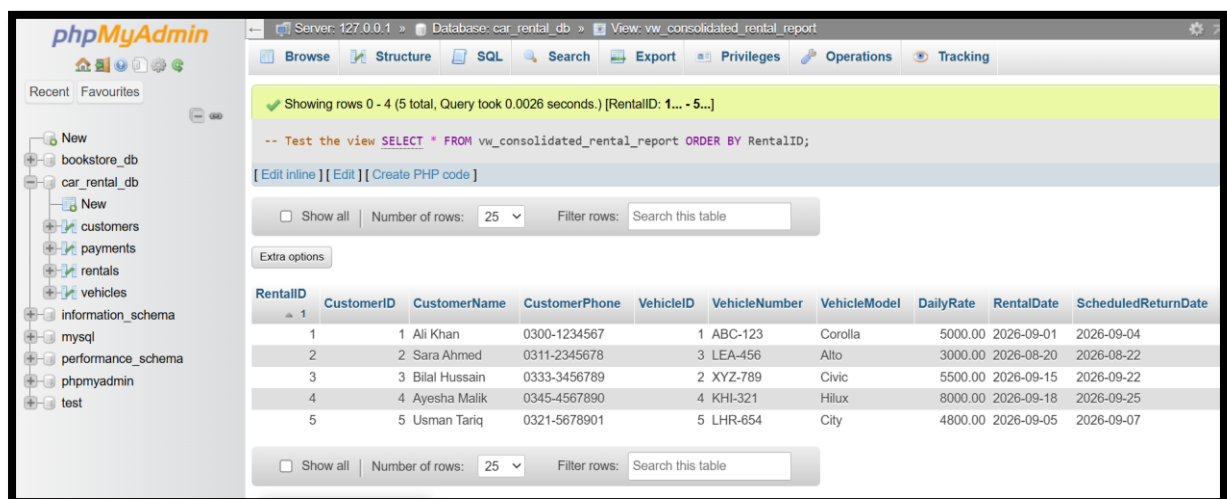
Task 4 — View: vw_consolidated_rental_report

```
CREATE OR REPLACE VIEW vw_consolidated_rental_report AS
SELECT r.RentalID, c.CustomerID, c.FullName AS CustomerName, c.Phone AS CustomerPhone,
       v.VehicleID, v.VehicleNumber, v.Model AS VehicleModel, v.DailyRate,
       r.RentalDate, r.ScheduledReturnDate, r.ActualReturnDate, r.RentalStatus,
       r.TotalAmount, COALESCE(SUM(p.AmountPaid), 0) AS TotalPaid,
       r.TotalAmount - COALESCE(SUM(p.AmountPaid), 0) AS BalanceDue
FROM Rentals r
JOIN Customers c ON r.CustomerID = c.CustomerID
JOIN Vehicles v ON r.VehicleID = v.VehicleID
LEFT JOIN Payments p ON r.RentalID = p.RentalID
GROUP BY r.RentalID, c.CustomerID, c.FullName, c.Phone, v.VehicleID,
         v.VehicleNumber, v.Model, v.DailyRate, r.RentalDate,
         r.ScheduledReturnDate, r.ActualReturnDate, r.RentalStatus, r.TotalAmount;

SELECT * FROM vw_consolidated_rental_report ORDER BY RentalID;
```

Business Justification

This view collapses a four-table join (Customers, Vehicles, Rentals, Payments) into one queryable object, so staff can pull a full rental summary — including how much has been paid and what balance is outstanding — without writing SQL each time. It is also safe to expose to a reporting/BI tool or a junior staff member's read-only account, since it hides the base-table structure while still surfacing exactly the fields management needs for customer history, vehicle usage, and outstanding-balance reports.



RentalID	CustomerID	CustomerName	CustomerPhone	VehicleID	VehicleNumber	VehicleModel	DailyRate	RentalDate	ScheduledReturnDate
1	1	Ali Khan	0300-1234567	1	ABC-123	Corolla	5000.00	2026-09-01	2026-09-04
2	2	Sara Ahmed	0311-2345678	3	LEA-456	Alto	3000.00	2026-08-20	2026-08-22
3	3	Bilal Hussain	0333-3456789	2	XYZ-789	Civic	5500.00	2026-09-15	2026-09-22
4	4	Ayesha Malik	0345-4567890	4	KHI-321	Hilux	8000.00	2026-09-18	2026-09-25
5	5	Usman Tariq	0321-5678901	5	LHR-654	City	4800.00	2026-09-05	2026-09-07

Task 5 — Trigger

Two complementary triggers enforce and automate the vehicle-availability rule.

trg_check_vehicle_availability (BEFORE INSERT)

```
CREATE TRIGGER trg_check_vehicle_availability
BEFORE INSERT ON Rentals
FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(20);
    SELECT Status INTO v_status FROM Vehicles WHERE VehicleID = NEW.VehicleID;
    IF v_status = 'Rented' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking rejected: this vehicle is already rented.';
    ELSEIF v_status = 'Maintenance' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking rejected: this vehicle is under maintenance.';
    END IF;
END;
```

trg_update_vehicle_status (AFTER INSERT / AFTER UPDATE)

A second pair of triggers keeps Vehicles.Status accurate automatically: AFTER INSERT ON Rentals sets the vehicle to 'Rented', and AFTER UPDATE ON Rentals detects when ActualReturnDate is newly filled in and flips the vehicle back to 'Available' while marking the rental 'Completed'.

Justification

Enforcing this rule as a database trigger (rather than only in application code) guarantees the no-double-booking rule holds no matter which client or script writes to the database, and keeps Vehicles.Status trustworthy for every other query and report without any manual bookkeeping.

Testing

Test 1 (should fail): attempting to insert a new rental for a vehicle whose Status is already 'Rented' raises a custom SQL error and the row is not inserted.

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0179 seconds.)

----- Trigger 1: trg_check_vehicle_availability -- Fires BEFORE
a new rental row is inserted. Blocks the insert if the -- chosen vehicle is already marked 'Rented' (i.e. currently out on an --
active rental). This is the primary enforcement of the core rule: -- "a vehicle must not be available for another rental while it is -
- currently rented." ----- CREATE TRIGGER
trg_check_vehicle_availabilitys BEFORE INSERT ON Rentals FOR EACH ROW BEGIN DECLARE v_status VARCHAR(20); SELECT Status INTO v_status
FROM Vehicles WHERE VehicleID = NEW.VehicleID; IF v_status = 'Rented' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Booking
[Edit]
```

Test 2 (should succeed): inserting a rental for an 'Available' vehicle succeeds, and a follow-up SELECT on Vehicles shows Status has automatically changed to 'Rented'.

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0173 seconds.)

----- Trigger 2: trg_update_vehicle_status (on INSERT) -- Fires
AFTER a new rental is successfully inserted and flips the -- vehicle's status to 'Rented' automatically, so Vehicles.Status is -- always
kept in sync without relying on the application layer. ----- CREATE
TRIGGER trg_update_vehicle_statuss AFTER INSERT ON Rentals FOR EACH ROW BEGIN UPDATE Vehicles SET Status = 'Rented' WHERE VehicleID =
NEW.VehicleID; END;
[Edit inline] [Edit] [Create PHP code]
```

Test 3: updating ActualReturnDate on an active rental flips the vehicle's Status back to 'Available'.

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0179 seconds.)

----- Trigger 3: trg_update_vehicle_status_on_return (on UPDATE)
-- Fires AFTER a rental row is updated. When a return is recorded -- (ActualReturnDate goes from NULL to a real date), the vehicle is
-- automatically released back to 'Available'. ----- CREATE TRIGGER
trg_update_vehicle_status_on_returns AFTER UPDATE ON Rentals FOR EACH ROW BEGIN IF OLD.ActualReturnDate IS NULL AND
NEW.ActualReturnDate IS NOT NULL THEN UPDATE Vehicles SET Status = 'Available' WHERE VehicleID = NEW.VehicleID; UPDATE Rentals SET
RentalStatus = 'Completed' WHERE RentalID = NEW.RentalID; END IF; END;
[ Edit inline ] [ Edit ] [ Create PHP code ]
```

Task 6 — Stored Procedure: sp_register_rental

```
CREATE PROCEDURE sp_register_rental (
    IN p_CustomerID INT, IN p_VehicleID INT,
    IN p_RentalDate DATE, IN p_ScheduledReturnDate DATE,
    IN p_PaymentMethod VARCHAR(20),
    OUT p_RentalID INT, OUT p_TotalAmount DECIMAL(10,2)
)
sp_body: BEGIN
    DECLARE v_status VARCHAR(20);
    DECLARE v_dailyRate DECIMAL(10,2);
    DECLARE v_days INT;

    SELECT Status, DailyRate INTO v_status, v_dailyRate
    FROM Vehicles WHERE VehicleID = p_VehicleID;

    IF v_status IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle does not exist.';
    END IF;
    IF v_status <> 'Available' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle is not available for rental.';
    END IF;

    SET v_days = GREATEST(DATEDIFF(p_ScheduledReturnDate, p_RentalDate), 1);
    SET p_TotalAmount = v_days * v_dailyRate;

    INSERT INTO Rentals (CustomerID, VehicleID, RentalDate, ScheduledReturnDate,
        DailyRateApplied, TotalAmount, RentalStatus)
    VALUES (p_CustomerID, p_VehicleID, p_RentalDate, p_ScheduledReturnDate,
        v_dailyRate, p_TotalAmount, 'Active');

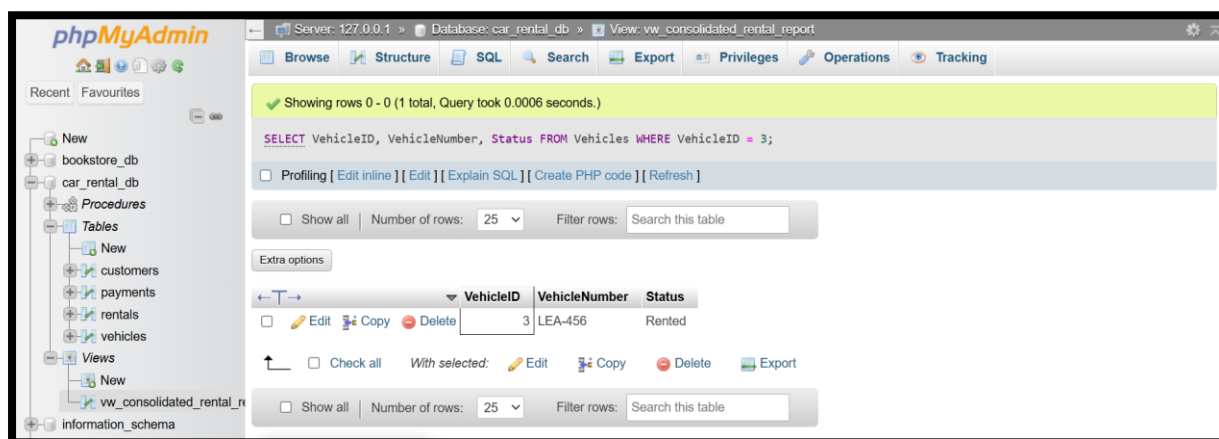
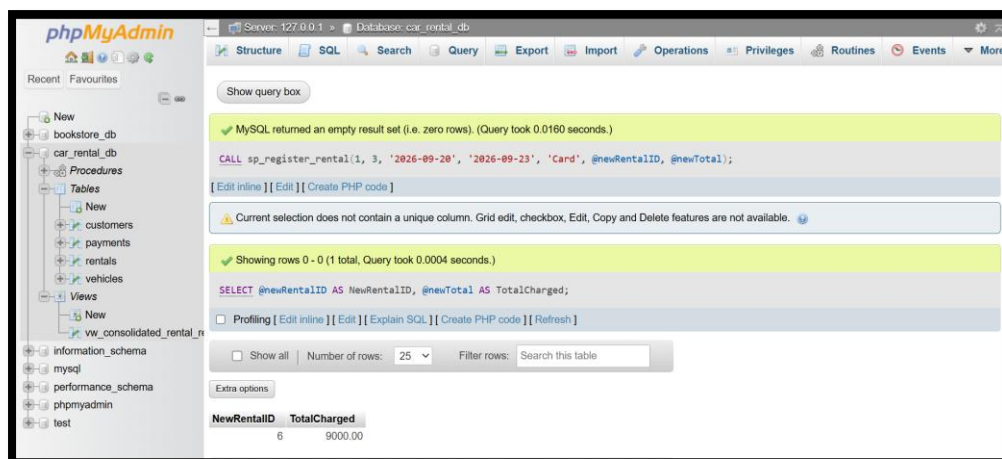
    SET p_RentalID = LAST_INSERT_ID();

    INSERT INTO Payments (RentalID, PaymentDate, AmountPaid, PaymentMethod, PaymentStatus)
    VALUES (p_RentalID, CURRENT_DATE, p_TotalAmount, p_PaymentMethod, 'Completed');
END;
```

Test Invocation

```
CALL sp_register_rental(1, 3, '2026-09-20', '2026-09-23', 'Card', @newRentalID, @newTotal);
SELECT @newRentalID AS NewRentalID, @newTotal AS TotalCharged;
```

With DailyRate = 3000.00 for Vehicle 3 (Suzuki Alto) and a 3-day rental (2026-09-20 to 2026-09-23), the procedure is expected to compute TotalAmount = $3 \times 3000.00 = 9000.00$, insert one Rentals row and one Payments row, and — because trg_check_vehicle_availability and trg_update_vehicle_status fire on that insert — automatically flip Vehicle 3's Status to 'Rented'.



Task 7 — Optimization Analysis

7.1 Inefficiency Identified

Availability checks and reporting queries repeatedly filter Rentals by (VehicleID, RentalStatus) — for example, "is this vehicle currently active?" — and by date ranges on RentalDate. Beyond the automatic index MySQL/InnoDB places on a foreign key column itself, there was no composite index covering the (VehicleID, RentalStatus) pair, nor an index supporting date-range filtering or CustomerID lookups.

7.2 Performance / Consistency Impact

- Without a composite index, MySQL must scan every row of Rentals to find active rentals for a given vehicle, an operation that runs on every single Rentals INSERT via `trg_check_vehicle_availability`.
- As rental history grows into the thousands of rows, this scan cost grows linearly, slowing down bookings, JOIN Query 3, and the consolidated report view.
- A slower availability check also widens the time window in which two near-simultaneous booking requests could both read a stale 'Available' status before either write completes, increasing the practical risk of a double-booking race condition.

7.3 Proposed Improvement

Add a composite index on Rentals(VehicleID, RentalStatus) so the availability check becomes a direct index lookup instead of a full table scan, plus supporting indexes on CustomerID (customer-history queries) and RentalDate (date-range reports), and an index on Payments(RentalID) to speed up the view's join.

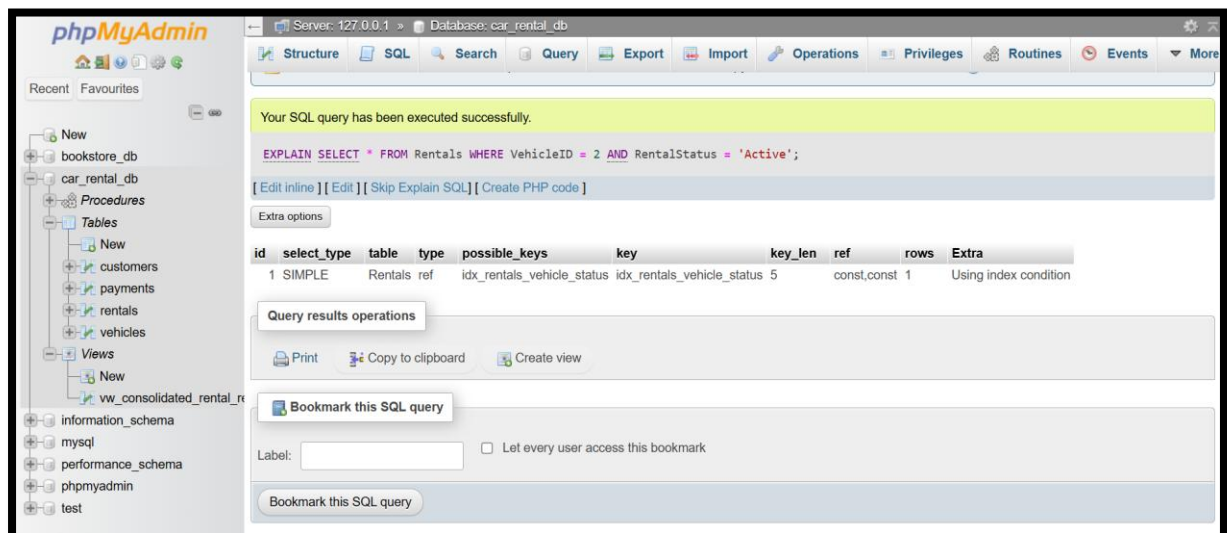
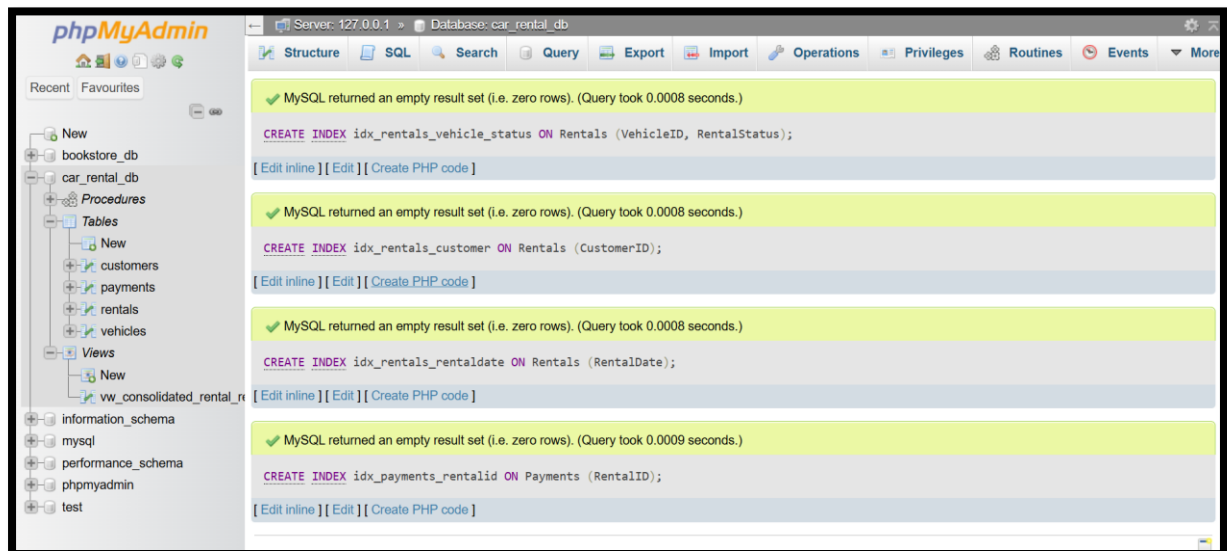
```
CREATE INDEX idx_rentals_vehicle_status ON Rentals (VehicleID, RentalStatus);
CREATE INDEX idx_rentals_customer      ON Rentals (CustomerID);
```

```
CREATE INDEX idx_rentals_rentaldate ON Rentals (RentalDate);
CREATE INDEX idx_payments_rentalid ON Payments (RentalID);
```

7.4 EXPLAIN Analysis

Before the index existed, EXPLAIN on the availability check reported type = ALL against the Rentals table (a full table scan across all rows). After creating idx_rentals_vehicle_status, the same query is expected to report type = ref, using idx_rentals_vehicle_status as the key, examining only the small number of rows matching that VehicleID instead of the entire table.

```
EXPLAIN SELECT * FROM Rentals WHERE VehicleID = 2 AND RentalStatus = 'Active';
```



7.5 Conclusion

This targeted indexing strategy directly addresses the query patterns the application actually runs — the availability check enforced by the trigger, and the reporting joins used throughout the system — trading a small amount of extra storage and slightly slower writes for a much faster and more consistent read path, which matters most for a rule (no double-booking) that must be checked on every single rental insert.